Sovereign streaming

Self hosting & Operational Excellence
for Streaming Infrastructure

Pavan Keshavamurthy
05/Nov/2026



Pavan Keshavamurthy

WORK

platformatory.io

streamtime.ai

leafcuttr.io

INTERESTS

Databases

Streaming

Infrastructure

Indian Classical Music

Population Genomics

OPEN SOURCE & COMMUNITY

Apache Kafka — Confluent Community Catalyst

Flink · OpenResty · Envoy · Seastar · DPDK

bengaluru.streams

meetup.com/bangalore-streams

p99 Bangalore

luma.com/p99-India

Bengaluru Distributed Systems & Databases

luma.com/systems.streams.bengaluru

Maxims for a sovereign streaming PaaS

01 **Sovereignty means control over dependencies and operating choices.**

02 **A private PaaS can optimize deeply for one company.**

03 **Expose workload intent. Translate provider vocabulary underneath.**

04 **Kubernetes provides the common execution substrate.**

05 **One management contract. Durable workflows underneath.**

06 **Day 2 is the product.**

07 **Design the infrastructure API for human and agent operators.**

08 **Air-gap changes topology while preserving the management contract.**

A lot of the world’s kafka is still unmanaged

- Regulated industries keep substantial streaming estates inside their own guardrails.
- On-prem, colo and long-tail clouds can be the cheapest place to move sustained GB/s.
- Users still expect the cloud operating model: API provisioning, upgrades, guardrails and self-service.

The tension

Cloud-like operations inside private infrastructure.

> **The cloud operating model is now expected inside private infrastructure.**

Three dimensions of sovereignty

Where it runs

Infrastructure sovereignty

on-prem · BYOC · edge · air-gap

Who controls operations

Operational sovereignty

provision · upgrade · recover · troubleshoot

What can be replaced

Dependency sovereignty

identity · certs · models · licenses · agents

Sovereignty comes from control over infrastructure, operations and dependencies.

A large PaaS optimizes for everyone. A private one can optimize for you.

PUBLIC / MULTI-CUSTOMER PAAS

bank · retail · gaming · gov · startup

↓

shared operating envelope

portable defaults · broad guardrails · common features

COMPANY-SCOPED PLATFORM

one trust boundary · known workloads · known constraints

↓

optimize vertically

hardware · topology · storage · runtime · operations

> **One underrated form of sovereignty: freedom to optimize.**

The estate becomes combinatorial

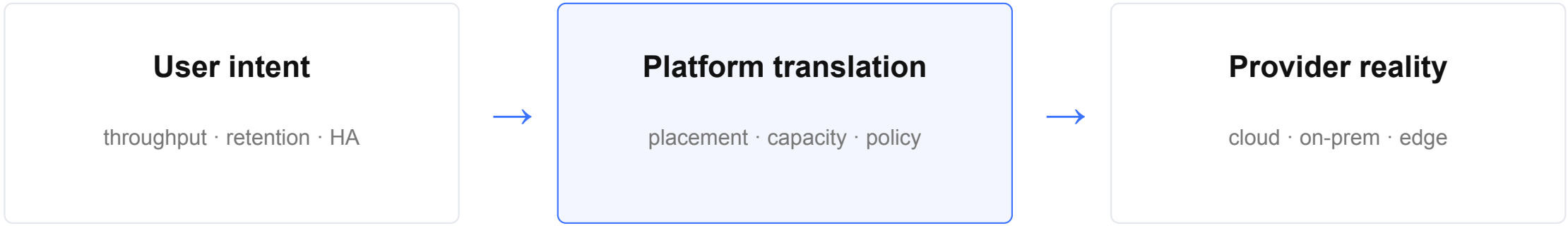
3 environments	×	**4** Kafka distributions	×	**5** lifecycle operations	×	**3** identity models	×	**4** infrastructure patterns

720 operational paths

Nobody wants 720 runbooks.

The hard part starts above the Kafka cluster

The job is to make many ways of running Kafka behave like one product.



> **The translation layer has to survive the whole lifecycle, including Day 2.**

Expose workload intent. Translate provider vocabulary underneath.

2 GB/s throughput · 7 day retention · 3 zone availability



capacity · placement · storage · JVM · network · provider SKU · operator settings

Users ask for an outcome. The platform owns the translation.

Kubernetes = common execution model

- Operators give us reconciliation and a portable control surface.
- Storage, load balancers, IAM, DNS, certificates and upgrade mechanics still differ.
- Air-gapped and edge environments make those differences explicit.



Preserve the differences that actually matter.

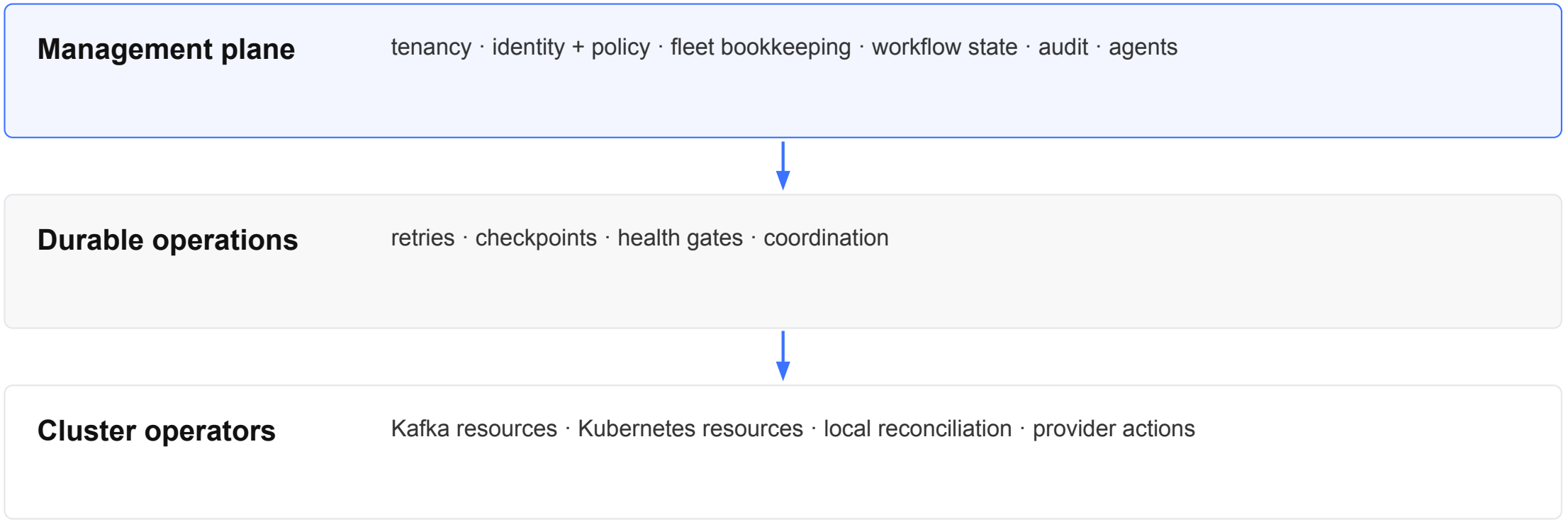
Long-running infrastructure work needs durable workflows



Create, delete, scale, upgrade and recover through one contract.

> **Infrastructure operations span retries, ordering, checkpoints and recovery.**

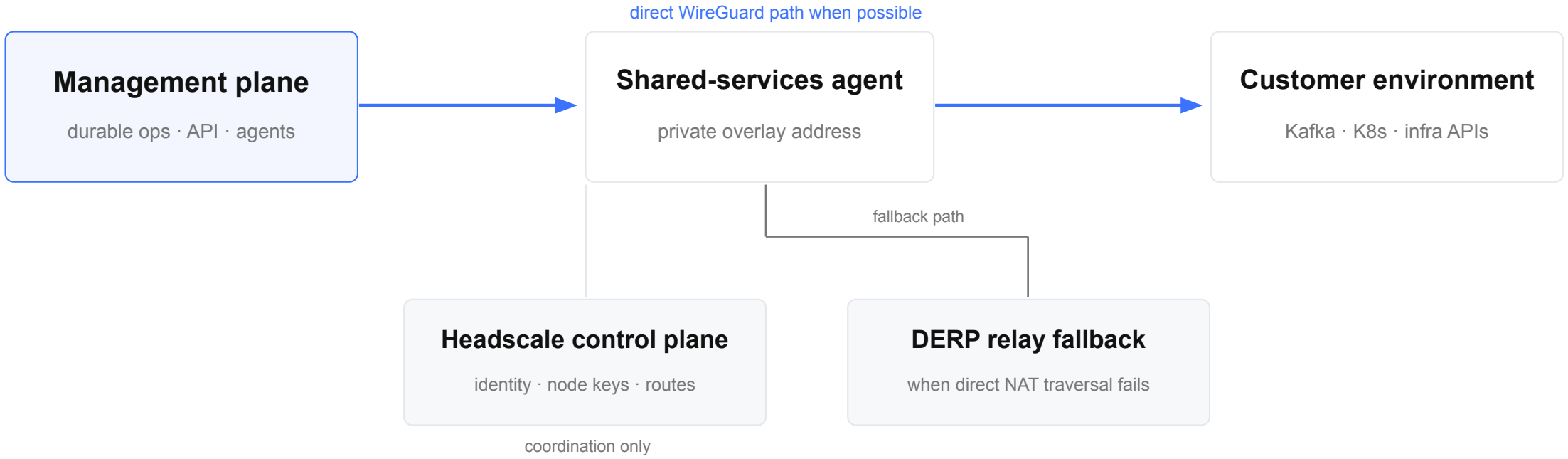
The management plane manages intent across time



The management plane carries intent across clusters and across time.

Private networking: keep management connectivity private

The shared-services agent exposes internal cluster and infrastructure APIs to the management plane over a private overlay.



Headscale coordinates the mesh. WireGuard carries the traffic. DERP is the encrypted relay path when peers cannot connect directly.

True air-gapping moves the control plane inside the boundary

Same APIs, operators and workflows. Different placement of the management plane and dependencies.

CONNECTED / BYOC	RESTRICTED EGRESS	TRUE AIR-GAP
Hosted management plane	**Hosted management plane**	**Management plane inside boundary**
Private Headscale overlay	Outbound-only private connectivity	Local control + local shared services
Customer Kubernetes + Kafka	Customer Kubernetes + Kafka	Customer Kubernetes + Kafka
normal artifact / model access	mirrors + policy-controlled egress	offline bundles + local registry / models

A true air-gap pulls every required control-plane dependency inside: artifacts, registry, models, identity and management services.

Day 2 is the product

Scale

up · out · in

Roll

config · certs

Upgrade

Kafka · operator

Recover

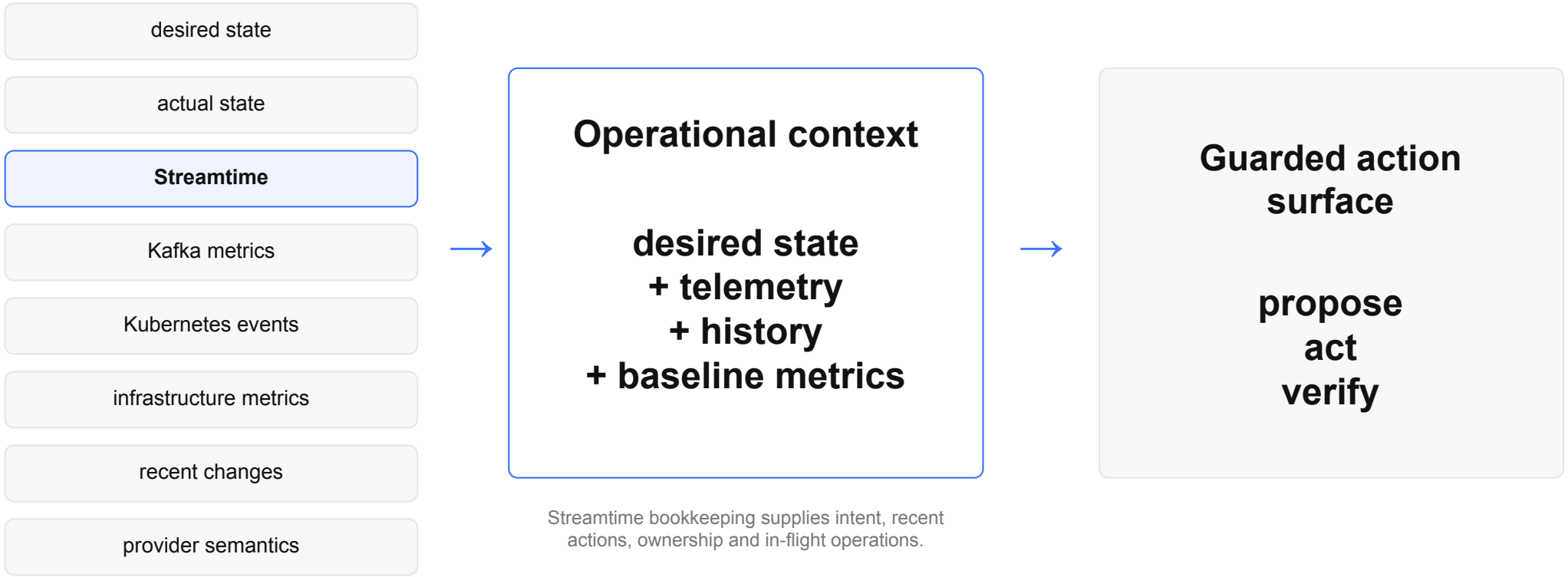
self-heal · DR

ordering **health gates** **retries** **pause points** **audit**

These are durable workflows with explicit state and recovery.

Build the infrastructure stack for human and agent operators

A growing share of infrastructure work will be delegated to agents. That changes what the platform API needs to expose.



Streamtime bookkeeping supplies intent, recent actions, ownership and in-flight operations.

> **Agents become first-class operators when context and safe actions share one API contract.**

Bring your own everything

certificates

your PKI

identity

your auth provider

distribution

OSS or commercial

license

your contract

models

your choice

agents

your policy

Keep the contract opinionated while dependencies remain replaceable.

Then bolt on the rest of the real-time stack

Kafka

Kafka Connect

Flink

+

One fleet, identity and operations model.

What we ended up building

Self-service contract	**tenancy · policy · fleet view · agents**
Durable operations API	meta-operator · provider-aware CRs
Real-time stack	Kafka · Flink · Kafka Connect · ecosystem bolt-ons
Your substrate	your Kubernetes · your infrastructure · your licenses

Streamtime

Thank you

streamtime.ai docs.streamtime.ai

Try Streamtime today

```
docker run -d --name streamtime --privileged \
  -p 3000:3000 -p 8443:8443 -p 7999:7999 \
  ghcr.io/streamtime-ai/streamtime:latest
```